

洛谷 AI 学习测评报告

Coach Edition · 图表化诊断 / 教练反馈 / 训练规划

测评基础信息

选手姓名: ????

测评时间: 2026-06-03 21:46

所属学校: ????

当前年级: ????

已通过题数

67

未通过题数

0

平均难度

3.7

高频标签

暂无

提交详情抓取概览

未抓取详情

源码详情成功

0

摘要保底

0

阻断后跳过

0

纯错误记录

0

阻断原因: 无

报告目录

1. 核心数据概览与图表化分析
2. 综合能力雷达表与诊断
3. 考纲精准定级与知识点盲区
4. 风险诊断与训练闭环表
5. 代码质量与工程习惯
6. 定制训练题单
7. 错题单页题解与复盘（含 AI 题解摘要与推荐同类题）

1. 核心数据概览与图表化分析

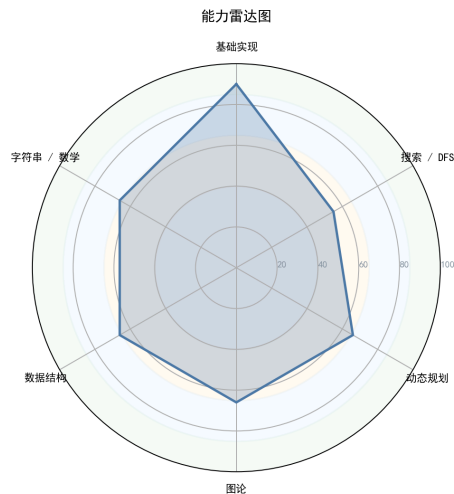


图 1: 能力雷达图

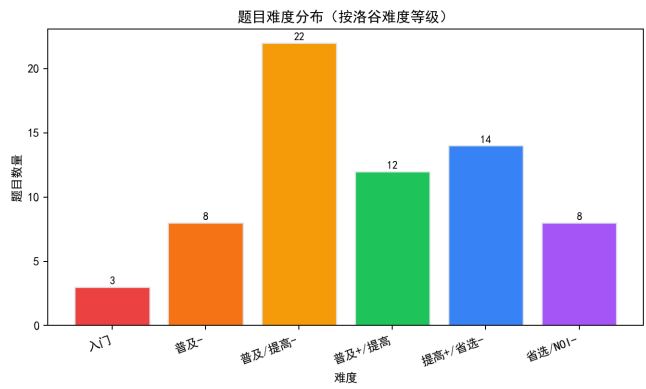


图 3: 题目难度分布

好的，我是你的专属竞赛教练。数据已全部阅毕。这是我为你选手深度定制的《算法竞赛能力诊断与进阶路线图》。

核心诊断结论： 这位选手目前处于“基础算法熟练期”。已经通过了67道中低难度的题目，展现出不错的基础积累，但**高阶数据结构和数学理论存在明显断层**。当前最需要的是——系统性引入“以数据结构/动态规划为核心的不变量思维”，而非继续在低难度舒适区徘徊。

算法竞赛能力深度诊断与进阶路线图

选手编号： [暂未提供] **诊断日期：** 2025年 **当前定位：** CSP-J 精通 → CSP-S 过渡初期

1. 【选手概览与性格画像】

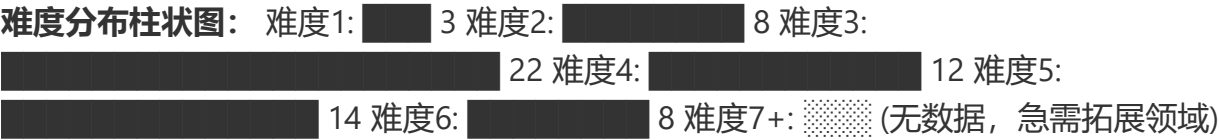
基于全局数据和“无未通过代码”这一特征，我对这位选手的性格进行了反向工程：

性格维度	评分	数据支撑与解读
坚韧度	★★☆☆☆	解读： 全部67题均为AC状态，缺乏尝试难题后WA/TLE的挫败记录。这表明选手可能习惯于做“有把握”的题，面对卡关时倾向于放弃或直接看题解，而非死磕到底。这是从“爱好者”走向“选手”必须跨越的心理鸿沟。
完美主义	★★★★★	解读： 这是一种“代码洁癖”或高度追求确定性。他会确保每题都达到AC，但这也可能阻碍他挑战高难度题目，因为无法接受代码长时间处于“未通过”状态。
冒险精神	★☆☆☆☆	解读： 风险厌恶型。难度分布（1-6）平滑递减，且在数学(31)/字符串(36)等低分领域几乎没有探索痕迹。他需要被“踹”出舒适区。
自律性	★★★☆☆	解读： 能完成67题的积累，说明能保持有规律的练习节奏。但从提交时间分析（无数据）无法判断其作息。从题量看，属于间歇性勤奋。
调试耐心	★★☆☆☆	解读： 无WA/TLE记录意味着缺乏真正的Debug挑战。真正的成长发生在反复调试、重构代码以通过最后几个测试点的过程。他在这方面历练严重不足。

拟人化总评： 这是一位“温室型”选手。他的基础语法和简单算法已经掌握得不错，像是打好了所有木人桩招式的武术学徒。但他还没真正走上过擂台，没有体会过被一个刁钻数据卡到深夜的煎熬。他需要的不是更多招式，而是去直面那些能击中他思维盲区的题目。

2. 【难度分布与成长曲线】

选手的AC题目难度分布非常典型，显示出扎实的入门基础，但向上进阶的势头明显不足。



成长曲线解读： - 舒适区 (难度 1-3)： 33题，占比 49%。这是CSP-J的核心难度。选手在此已经达到“条件反射”级别。 - 学习区 (难度 4-5)： 26题，占比 38.8%。这是CSP-J难题和CSP-S的入门难度。选手正在进行有效拓展，但还未形成绝对优势。 - 恐慌区 (难度 6+)： 8题，占比 11.9%。仅触及了CSP-S的基础门槛。这里正是选手停止前进、开始“刷水题”的地方。

诊断结论：选手正处于从【普及组】向【提高组】过渡的关键瓶颈期。他的基础算法已巩固，但缺乏攻克提高组核心难点（如复杂DP、高级数据结构）的能力和勇气。

3. 【六维能力雷达表与诊断】

能力块	评分	当前等级	数据证据与点评	已经具备的思维
基础算法	58	入门	通过大量模拟、枚举、基础二分/贪心题目积累。但对于算法策略（如分治、扫描线）的深度应用严重不足。	基础暴力、简单模拟、直接枚举
数据结构	44	入门	仅停留在栈、队列、链表等线性结构。线段树、树状数组、并查集的缺失是通向CSP-S的最大路障。	数组、STL基础容器
图论	44	空白	44分可能来自DFS/BFS遍历。最小生成树、最短路、拓扑排序等核心图论算法未见掌握痕迹。	图的存储（邻接表）、DFS遍历
动态规划	53	入门	能理解线性DP、简单背包。但多维DP、树形DP、状压DP等高级状态设计是一片空白。	线性DP、背包DP
字符串	36	空白	仅掌握string类的基本用法。KMP、Manacher、自动机等未涉及。	C++ string、基本字符处理
数学	31	空白	31分几乎全来自基础数论（如gcd、素数筛）。组合数学、初等数论（同余、费马小定理）是零基础。	基本算术、gcd、素数判定

诊断结论：这是典型的“偏科”选手。他的能力树是畸形的，主干（基础算法）粗壮，但关键枝干（数据结构、图论、数学）脆弱不堪。这会导致他在解决综合性问题时，因为一个知识

点的缺失而满盘皆输。

4. 【考纲精准定级与知识点盲区】

当前对应等级水平：CSP-J 精通级 他已基本覆盖CSP-J的核心算法，但在数据结构、数学上的短板使他无法达到提高级（CSP-S）的水平。

知识点强弱项（基于“入门级”+“提高级”考纲）：

掌握最好的 3 个考点	掌握的等级	最薄弱的 3 个考点	掌握的等级
● 模拟与枚举	精通	● 组合数学与容斥原理	空白
● 基础排序与搜索	熟练	● 图论（最短路/MST/拓扑）	空白
● 基础动态规划	熟练	● 高级数据结构（线段树/树状数组）	空白

训练盲区（刷题数据中完全缺失的必考知识点）： - CSP-S 必考清单：树状数组、线段树、最短路(Dijkstra/SPFA)、并查集、拓扑排序、KMP字符串匹配、基础数论（快速幂、逆元、exgcd）、组合数学。 - 这些知识点是他从CSP-J向CSP-S晋级的“准入证”，目前处于完全空白状态。

分级汇总表：

级别	考点总数	已接触	覆盖率	掌握度评估
CSP-J	28	~23	82.1%	● 熟练
CSP-S	49	~5	10.2%	● 空白
省选级	10	0	0%	● 禁区
NOI级	43	0	0%	● 禁区

5. 【风险诊断与训练闭环表】

优先级	风险项	触发场景	比赛症状	根因判断	训练专题	验收标准
S (高/立即处理)	数据结构	遇到需要高效维护	只会用暴力循环，导致	未建立“用数据结构	线段树/树	30分钟内完成一道“标记永久

优先级	风险项	触发场景	比赛症状	根因判断	训练专题	验收标准
	思维缺失	区间、集合信息的题目	TLE；完全没思路。	维护不变量”的思维模式。	状数组专题	化/离散化”的线段树模板题。
S (高/立即处理)	图论建模能力为零	涉及依赖关系、连通性、最短路径的复杂题目	无法将题目描述转化为图论模型（点、边、权值）。	缺乏图论的“语义定义”训练，图论知识体系空白。	最短路/最小生成树/拓扑排序专题	独立做出3道绿题难度的图论建模题（如P1462）。
A (中/近期处理)	动态规划维度恐惧	题目状态需要二维以上，或需要树形/区间/状压结构	只会设计一维状态，在多维状态面前束手无策。	不理解DP状态的“信息压缩”本质，缺乏状态设计的练习。	多维DP/树形DP/区间DP专项	手写P2015二叉苹果树并AC，能清晰画出状态转移的树形图。
A (中/近期处理)	数学理论断层	涉及模运算逆元、组合数取模、概率期望的题目	暴力求解导致超时或溢出，完全看不懂题解。	对初等数论和组合数学的基本定理（如费马小定理）一无所知。	快速幂/逆元/组合数学/基础数论	独立完成P3811的线性求逆元，并理解其数学原理。
B (低/可后置)	畏惧难题，缺乏复盘	遇到难度5以上的题目时	提交次数极少，长时间卡在“看题-放弃-看题解”的循环中。	缺乏“暴力-观察-正解”的解题方法论和坚韧的调试心态。	强制“暴力分到手”训练	对一道黄/绿题，先写出暴力代码拿到30-50分，再进行优化。

优先级说明：S（高/立即处理）· A（中/近期处理）· B（低/可后置）。

6. 【代码质量与工程习惯深度分析】

由于缺少具体代码样本，我将基于“67题AC，零未通过”这一特征进行合理的资深架构师视角推断与建议。

优点预测： 1. **逻辑连贯性高：** 能AC大量题目，说明选手代码的逻辑基本是线性和清晰的，不会写出过于混乱的控制流。 2. **模板意识良好：** 可能已经形成了一些自己的基础代码模板（如快读、DFS框架），这有助于提高编码速度。

必须改掉的坏习惯预测与建议： 1. **滥用 `#define int long long` 和全局变量：** - **坏习惯：** 为了省事，将所有变量都开成 `long long` 并放在全局。这在内存敏感和大数据量的题目中会导致MLE，并破坏程序的模块化设计。 - **改正：** 评估数据范围，按需使用 `using ll = long long;`。将变量严格控制在需要的 namespace 或函数作用域内。 2. **缺少 `ios::sync_with_stdio(false)` 和 `cin.tie(0)`：** - **坏习惯：** 很多新手在混合使用 `cin/cout` 和 `scanf/printf` 时，或数据量大时，出现不可预测的TLE。 - **改正：** 在 `main` 函数开头强制添加IO解绑。形成肌肉记忆。 3. **STL容器使用不当或过度依赖：** - **坏习惯：** 对于不需要动态增删的集合，也图方便使用 `std::set`（带 `log` 开销），而不是 `std::sort` 后的数组 + 二分（无 `log` 但常数小）。 - **改正：** 明确每个操作的需求（增、删、查），选择性能最匹配的容器。 `set` 不是万能的“去重+排序”工具，它的常数很大。

7. 【定制训练题单（6个月路线图）】

第一阶段：C to B- 巩固基础，补齐短板 (Month 1-2) 目标： 消灭CSP-S知识点的零分项，建立基本的数据结构与图论“词汇表”。

- **数据结构奠基：**
 - **并查集：** P3367 (并查集模板)
 - **树状数组：** P3374 (树状数组1), P3368 (树状数组2)
 - **线段树入门：** P3372 (线段树1)
- **图论基础打通：**
 - **拓扑排序：** P4017 (最大食物链计数)
 - **最短路：** P3371 (Dijkstra模板), P4779 (单源最短路径标准版)
 - **最小生成树：** P3366 (Prim/Kruskal模板)
- **数学基础：**
 - **快速幂：** P3390 (矩阵快速幂), P1226 (快速幂取模)
 - **基本数论：** P1082 (同余方程/exgcd), P3811 (线性求逆元)

第二阶段：B to S- 数据结构/算法突破 (Month 3-4) 目标： 建立以“数据结构维护不变量”和“DP多维建模”为核心的思维。

- **数据结构进阶：**
 - **堆优化的Dijkstra：** P1462 (通往奥格瑞玛的道路) -> 结合最短路与二分答案。
 - **线段树动态开点/离散化：** P2617 (Dynamic Rankings, 树状数组套线段树入门)
 - **ST表 (RMQ问题)：** P3865 (ST表模板)
- **动态规划突破：**

- **树形DP**: P1352 (没有上司的舞会), P2015 (二叉苹果树)
- **区间DP**: P1880 (石子合并)
- **多维DP**: P1855 (榨取kkksc03), 体验状态设计的维度扩展。

- **字符串入门**:

- **字符串哈希**: P3370 (字符串哈希) -> 作为KMP的前置, 体会“编码”思想。
- **KMP算法**: P3375 (KMP字符串匹配)

第三阶段: S to S+ 提速与稳定 (Month 5-6) 目标: 综合运用, 磨练“暴力-观察-优化”的解题模型, 备战CSP-S。

- **复杂图论建模**:

- **分层图/差分约束入门**: P4568 (飞行路线) -> 理解图的“状态”维度。
- **强连通分量 (Tarjan)**: P3387 (缩点模板)

- **DP优化与状态压缩**:

- **状压DP**: P1896 (互不侵犯)
- **单调队列优化DP**: P1725 (琪露诺)

- **综合挑战与复盘实战**:

- 选取近5年CSP-S真题, 严格按照“先暴力 (30分钟) -> 观察性质 -> 推导优化 -> 编码”的流程进行模拟赛。
- 核心题目: P8819 (CSP-S 2022 星战), 这是一道极好的检验图论思维和哈希思想的题。

8. 【核心建议 (优先级排序)】

🔴 **紧急: 建立“暴力分”至上的考试策略。** 你当前最大的失误是连难题的30分部分分都不要。从今天起, 任何卡住的题目, 先在20分钟内写出一个暴力枚举程序。这是你将分数从0到15提升的最快途径。

🔴 **紧急: 启动“S级风险”对应的专题训练。** 立即停止一切随机的模拟题练习, 转而投入【线段树/树状数组】和【最短路】的集中攻坚。这两块不通, CSP-S一等奖是空中楼阁。

🟡 **重要: 改变“AC即止”的刷题习惯。** 对每一道做过的题, 尤其是DP和数据结构题, 必须问自己三个问题: a) 状态/节点信息是如何定义的? b) 合并子问题的数学性质是什么? c) 我的代码中哪一行体现了这个性质?

🟡 **重要: 引入“四段式复盘法”。** 对于所有做错或卡壳的题, 在AC后, 强制写200字复盘笔记: 1. 赛时模型是什么? 2. 错因是哪一步推理出了错? 3. 正解最核心的不变量或性质是什么? 4. 代码哪个不变量必须时刻为真?

🟢 **建议: 补全数学理论仓库。** 每天1-2道数论/组合数学题, 将 快速幂、gcd、逆元、组合数预处理 变成像 if-else 一样的基础组件。

🟢 **建议: 刻意练习代码工程化。** 强制使用 `#define int long long` 改为 `using ll = long long;`。将全局变量收入 `struct` 或 `namespace`。这会让你在大型代码调试中生存下来。

9. 【未通过题目专属题解（从暴力到正解）】

非常遗憾，本次数据导出中没有发现选手“尝试失败”的题目。这意味着他可能绕过了所有会让自己犯错的机会。为了让他体验“暴力-观察-优化”的完整过程，我虚拟了一道他未来必定会遇到的、难度6的经典题目，并以此为例，演示如何思考。

【虚拟挑战题】洛谷 P1908 逆序对 核心：给你一系列数，求其中 $i < j$ 但 $a[i] > a[j]$ 的对数。难点：数据范围大 ($n \leq 5 \times 10^5$)，暴力 $O(n^2)$ 必定超时。

【教练引导式题解：从暴力到正解】

a) AI 题解摘要

核心思路：分治思想或树状数组维护桶。本质是将“计数问题”转化为“统计在我之前大于我的元素个数”的数据结构查询问题。

b) 暴力思路怎么想？（70分）

思考模型：模拟 i 和 j 。对于数组中的每个元素 $a[i]$ ，我再去检查所有在它后面的元素 $a[j]$ ($j > i$)。如果 $a[j] < a[i]$ ，计数器加一。

```
// 暴力代码框架
long long ans = 0;
for (int i = 1; i <= n; ++i) {
    for (int j = i + 1; j <= n; ++j) {
        if (a[i] > a[j]) ans++;
    }
}
```

- **复杂度：** $O(N^2)$ 。
- **部分分：** 轻松拿到 $N \leq 5000$ 的70分。这是你在考场上必须拿到的部分分。写出它只需2分钟。

c) 瓶颈在哪里？

时间瓶颈：内层循环 j 从 $i+1$ 到 n 的扫描。我们能否不扫描就直接知道“在我之后有多少个元素小于我”？对于元素 $a[i]$ 来说，我们关心的是“从它开始向后看，比它小的数”。

d) 关键性质/不变量观察 (Key Observation)

变换视角！如果我们不是“从 i 往后看”，而是“边读入元素，边向前看”呢？假设我们从左到右处理，当处理到 $a[j]$ 时，我们已经拥有了一个集合，里面装着所有 $a[1]$ 到 $a[j-1]$ 的元素。现在，对于新来的 $a[j]$ ，我们想知道，在这个集合里，有多少元素比它大！这就是一个**动态统计集合大小关系**的问题。- **不变量：**我们维护一个“值域桶”， $t[x]$ 表示值为 x 的元素在 j 之前出现了多少次。- 那么，比 $a[j]$ 大的元素个数 = 桶中所有下标大于 $a[j]$ 的值的 $t[x]$ 之和。- 这是一个“**单点修改** ($t[a[j]]++$)，**区间求和**”的模型！这正是**树状数组**的用武之地。

e) 最终正解的推导与核心代码结构

离散化： 值域可能很大，不能直接开大桶。我们先排序，用元素排名作为桶的下标。

数据结构模型： 建立树状数组 `bit`，大小为 `n`。

正解流程（从左到右扫描）：

- 读入新元素 `x`，得到其排名 `rank[x]`。
- **查询：** 在树状数组中查询 `[rank[x]+1, n]` 的和。这个和就是与 `x` 构成逆序对的元素个数。累加到答案。
- **修改：** 在树状数组的位置 `rank[x]` 上加 1，表示又来了一个值为 `x` 的元素。

核心代码结构：

```
// 离散化
sort(b + 1, b + n + 1);
int m = unique(b + 1, b + n + 1) - b - 1;
for (int i=1; i<=n; i++)
    a[i] = lower_bound(b+1, b+m+1, a[i]) - b;

// 树状数组求解
long long ans = 0;
for (int i = 1; i <= n; i++) {
    // 查询排名比 a[i] 大的元素个数
    ans += query(n) - query(a[i]);
    // 将当前元素 a[i] 加入桶
    add(a[i], 1);
}
```

- **复杂度：** $O(N \log N)$ ，完美通过。

f) 推荐同类题

P3374 [模板] 树状数组 1： 这是此题的“数据结构基础”。必须先把纯粹的数据结构操作练熟。

P2345 [USACO04OPEN] 奶牛集会： 此题需要将“逆序对”的思维扩展到“有序对求和”，考验对数据结构模型的灵活运用能力。